

Project 1: RAG-Based Q&A Pipeline for RapidFire AI

CSE/DSC 234 — Data-Centric AI and AI Engineering, Spring 2026

Sukruth Rao
PID: A69042450

1 Data Creation Methodology

We created 20 golden-labeled Q&A pairs over the RapidFire AI OSS documentation which covered five questions per four question types.

Question types:

- **Factual Questions** — Questions with single, directly retrievable answers.
Example: “What vector store does RapidFire AI use by default?”
- **Conceptual Questions** — Questions requiring understanding of a system component or underlying idea.
Example: “How does RapidFire AI’s adaptive execution engine differ from traditional sequential execution?”
- **Procedural Questions** — Questions asking for steps or instructions to accomplish a task.
Example: “How do I configure a PGVector persistent store in Read mode?”
- **Comparative Questions** — Questions comparing multiple features, tools, or documentation sections.
Example: “What is the difference between RFGridSearch and RFRandomSearch?”

Reference answers were generated by an AI coding agent given the respective RST source. These were manually proofread and corrected where there was inaccuracy. The source evidence line spans were also verified manually by going through the respective RST files, and incorrect line ranges were manually corrected. We also created some questions that include multiple file source evidence for which were distributed in multiple documentation sections.

We distributed the questions across 12 RST files to cover a broad range of RapidFire AI concepts (RAG specs, experiment ops, online aggregation, IC ops, generators, etc). Questions that were simplistic, vague, or answerable without the corpus were identified and rewritten. We also verified that each reference answer is directly supported by the documentation.

2 Final Pipeline Architecture

Chunking

We use `RecursiveCharacterTextSplitter` with `chunk_size=1000`, `chunk_overlap=150`. The `add_start_index` flag provides the exact character offset for each chunk directly from the splitter for accurate line-number metadata. The files were pre-chunked and uploaded to Pinecone across namespaces (`chunk256`, `chunk512`, `chunk1000`) to perform efficient multi-config experimentation without re-embedding. Chunk size 1000 was selected based on multi-config experiments, which are explained in more detail in Section 3.

Embedding

We used TritonAI’s `api-tgpt-embeddings` for our preliminary runs, which resulted in extremely poor retrieval which resulted in a poor F1@5 score of around 0.06. Through the multi-config experimentation, we selected `BAAI/bge-small-en-v1.5` (HuggingFace) with normalized embeddings. Subsequently, we observed that this produced a 5 times improvement in the retrieval score.

Indexing and Retrieval

We store chunks Pinecone (persistent cloud vector store) under the `project1-rag-index` index. We use Pinecone in Read mode to help run 36 retrieval configurations without re-embedding the files each time. For the final pipeline, we query the `chunk1000` namespace. We use vector similarity search with $k = 5$. We observed that similarity with `chunk1000` consistently outperformed MMR and score-threshold through our multi-config RapidFire experiments (Section 4).

Reranking

After retrieving the top 5 candidates from Pinecone, we pass them through a cross-encoder reranker, cross-encoder/ms-marco-MiniLM-L-4-v2. We selected this over BGE-reranker-base as it achieves higher NDCG@5 and MRR while being lighter and faster (see Section 4). Embeddings generate separate vector representations and compare them through similarity scoring, which can potentially miss important contextual relationships.

Generation

We use `claude-sonnet-4.6` via TritonAI to produce answers with the prompt: *“You are a helpful assistant that answers questions based only on the provided context. Answer thoroughly and cite specific details from the context. Context: context Question: query Answer:.”* The `--generator-model` CLI argument allows for fallback to `api-gpt-oss-120b` if necessary.

The top 5 reranked chunks are concatenated with newline separators and passed as context to the generator. We can estimate that context occupies approximately 1,400–1,600 tokens, which is within the 2,000-token budget

3 Experimentation Methodology

We ran a systematic multi-config grid search with 36 configurations using RapidFire AI’s `RFGGridSearch`, which was written in `run_rf_evals.ipynb`.

Experimental Design

We pre-indexed the corpus into Pinecone at three chunk sizes (`chunk256`, `chunk512`, `chunk1000`) before running any experiments, which let us query Pinecone directly when testing different retrieval and reranking configurations. This way, we avoid re-embedding the documents which inherently helps keep costs down and results comparable across all configurations.

Knob	Values	# Values
Chunk size	256, 512, 1000	3
Search type	similarity $k=5$, MMR $k=5$, score threshold@0.5	3
Reranker	MiniLM-L-4-v2, BGE-reranker-base, none	3
Retriever type	Pinecone (dense), BM25 (sparse)	2
Total	27 dense + 9 sparse	36 configs

Table 1: RapidFire AI multi-config grid search (36 total configurations). All configs use BGE-small-en-v1.5 embeddings. Generator: `api-mistral-small-3.2-2506`.

We compute Token F1, Precision@5, Recall@5, F1@5, NDCG@5, and MRR for each configuration.

We used `api-mistral-small-3.2-2506` as the generator for all grid search experiments to minimize cost, and switched to `claude-sonnet-4-6` for the final pipeline to maximize generation quality.

Knob Logic:

- **Chunk size:** We wanted to test from small to large range to find the sweet spot between too fragmented (256) and diluted (1000).

- **Search type:** We use similarity search which retrieves the closest vectors. In addition, we tested with MMR to see if inducing diversity and reducing redundancy among results would help when chunks overlap. We used score threshold to cover all search modes that were supported by `RFLangChainRagSpec`
- **Reranker:** We tested two cross-encoder rerankers of different sizes: `MiniLM-L-4-v2` (lighter, faster) and `BGE-reranker-base` (heavier, more accurate) along with no reranker, so as to get an understanding of how much reranking actually helps on this corpus.
- **Retriever Type:** We compared dense retrieval with Pinecone and sparse retrieval with BM25 to determine whether semantic similarity or keyword based matching works better for the documentation corpus.

4 Results and Analysis

RapidFire AI Experiments Results

Retriever	Search	Reranker	TokF1	P@5	R@5	F1@5	NDCG@5	MRR	Ret. Score	Class Ret.
Pinecone/1000	similarity	MiniLM-L4	0.2792	0.2933	0.8833	0.4254	0.8801	0.9044	0.6258	0.5340
Pinecone/1000	score-thresh	MiniLM-L4	0.2305	0.2933	0.8833	0.4254	0.8801	0.9044	0.6258	0.5340
Pinecone/1000	similarity	BGE-reranker	0.2840	0.2933	0.8833	0.4254	0.8699	0.8722	0.6152	0.5340
Pinecone/1000	score-thresh	BGE-reranker	0.2832	0.2933	0.8833	0.4254	0.8699	0.8722	0.6152	0.5340
Pinecone/512	similarity	BGE-reranker	0.2362	0.2933	0.8889	0.4189	0.8510	0.8600	0.6058	0.5337
Pinecone/512	score-thresh	BGE-reranker	0.2337	0.2933	0.8889	0.4189	0.8510	0.8600	0.6058	0.5337
Pinecone/1000	MMR	BGE-reranker	0.2728	0.2178	0.8500	0.3393	0.8939	0.8944	0.5864	0.4690
Pinecone/512	similarity	MiniLM-L4	0.2442	0.2933	0.8889	0.4189	0.8290	0.8019	0.5858	0.5337
Pinecone/512	score-thresh	MiniLM-L4	0.2383	0.2933	0.8889	0.4189	0.8290	0.8019	0.5858	0.5337
Pinecone/1000	similarity	none	0.2850	0.2933	0.8833	0.4254	0.8126	0.7952	0.5816	0.5340
Pinecone/1000	score-thresh	none	0.2839	0.2933	0.8833	0.4254	0.8126	0.7952	0.5816	0.5340
Pinecone/1000	MMR	MiniLM-L4	0.2708	0.2178	0.8500	0.3393	0.8856	0.8822	0.5812	0.4690
Pinecone/512	similarity	none	0.2381	0.2933	0.8889	0.4189	0.7311	0.6619	0.5263	0.5337
Pinecone/512	score-thresh	none	0.2403	0.2933	0.8889	0.4189	0.7311	0.6619	0.5263	0.5337
Pinecone/1000	MMR	none	0.2645	0.2178	0.8500	0.3393	0.7798	0.7570	0.5235	0.4690
Pinecone/256	similarity	BGE-reranker	0.2054	0.2856	0.7500	0.3952	0.7034	0.6952	0.5198	0.4769
Pinecone/256	score-thresh	BGE-reranker	0.2102	0.2856	0.7500	0.3952	0.7034	0.6952	0.5198	0.4769
Pinecone/256	similarity	MiniLM-L4	0.2133	0.2856	0.7500	0.3952	0.6876	0.6726	0.5102	0.4769
Pinecone/256	score-thresh	MiniLM-L4	0.2078	0.2856	0.7500	0.3952	0.6876	0.6726	0.5102	0.4769
Pinecone/256	MMR	BGE-reranker	0.2175	0.2489	0.7722	0.3672	0.7080	0.6889	0.5032	0.4628
Pinecone/256	MMR	MiniLM-L4	0.2159	0.2489	0.7722	0.3672	0.7013	0.6759	0.4983	0.4628
Pinecone/512	MMR	BGE-reranker	0.2324	0.2178	0.7278	0.3246	0.6902	0.6648	0.4743	0.4234
Pinecone/256	similarity	none	0.2126	0.2856	0.7500	0.3952	0.6278	0.5807	0.4723	0.4769
Pinecone/256	score-thresh	none	0.2056	0.2856	0.7500	0.3952	0.6278	0.5807	0.4723	0.4769
Pinecone/512	MMR	MiniLM-L4	0.2270	0.2178	0.7278	0.3246	0.6710	0.6378	0.4628	0.4234
Pinecone/256	MMR	none	0.2047	0.2489	0.7722	0.3672	0.6315	0.5959	0.4609	0.4628
BM25/256	—	BGE-reranker	0.1852	0.2311	0.7000	0.3291	0.6360	0.6193	0.4538	0.4201
BM25/512	—	BGE-reranker	0.1887	0.2311	0.7000	0.3291	0.6360	0.6193	0.4538	0.4201
BM25/1000	—	BGE-reranker	0.1451	0.2311	0.7000	0.3291	0.6360	0.6193	0.4538	0.4201
BM25/256	—	MiniLM-L4	0.1897	0.2311	0.7000	0.3291	0.6232	0.6148	0.4496	0.4201
BM25/512	—	MiniLM-L4	0.1844	0.2311	0.7000	0.3291	0.6232	0.6148	0.4496	0.4201
BM25/1000	—	MiniLM-L4	0.1835	0.2311	0.7000	0.3291	0.6232	0.6148	0.4496	0.4201
Pinecone/512	MMR	none	0.2323	0.2178	0.7278	0.3246	0.6338	0.6026	0.4447	0.4234
BM25/256	—	none	0.1856	0.2311	0.7000	0.3291	0.5338	0.4774	0.3928	0.4201
BM25/512	—	none	0.1856	0.2311	0.7000	0.3291	0.5338	0.4774	0.3928	0.4201
BM25/1000	—	none	0.1972	0.2311	0.7000	0.3291	0.5338	0.4774	0.3928	0.4201

Table 2: All 36 configurations, ranked by Ret. Score = $\frac{1}{4}(\text{P@5} + \text{F1@5} + \text{NDCG@5} + \text{MRR})$. Class Ret. = $\frac{1}{3}(\text{P@5} + \text{R@5} + \text{F1@5})$. Pinecone configs use `BAAI/bge-small-en-v1.5` embeddings; generator: `api-mistral-small-3.2-2506`. Entry in bold is the final selected configuration.

We can observe F1@5 improved with increase in chunk size: 0.395 (chunk256) $\rightarrow$ 0.419 (chunk512) $\rightarrow$ 0.425 (chunk1000). Token F1 followed the same trend (0.217 $\rightarrow$ 0.244 $\rightarrow$ 0.285). At chunk=256, each chunk captures only a fragment of the documentation which didn’t contain enough context to answer a question completely. Chunk=512 is competitive but slightly underperforms chunk=1000 on both F1@5 and Token F1.

We also observe that MMR showed lower F1@5 than similarity search across all chunk sizes(e.g., chunk1000:

Metric	Best BM25	Best Pinecone
F1@5	0.3291	0.4254
NDCG@5	0.6360	0.8801
MRR	0.6193	0.9044
Token F1	0.1972	0.2850
ClsRet	0.4201	0.5340

Table 3: Best BM25 vs. Best Pinecone Retrieval Performance

similarity 0.4254 vs MMR 0.3393). This may be because the corpus is small enough that the top retrieved chunks are already covering different parts of the documentation, hence forcing diversity removes relevant results.

Score threshold (0.5) produced identical retrieval metrics to similarity search across all chunk sizes (e.g., chunk=1000: F1@5=0.4254, NDCG@5=0.8801, MRR=0.9044). This indicates that the similarity threshold had no effect on which chunks were returned.

For chunk1000 with similarity search, the three reranker configurations (MiniLM-L4, BGE-reranker, none) produced identical F1@5 (all 0.4254). Rerankers reorder the retrieved set but cannot change which documents are fetched, so F1@5 remains identical. Token F1 differences were ≤ 0.006 . We select MiniLM-L4 as the reranker in the final pipeline as it achieves higher NDCG@5 (0.8801 vs. 0.8699) and MRR (0.9044 vs. 0.8722) than BGE-reranker while being lighter and faster.

Final pipeline evaluation

Metric	Validation Set	Golden Q&A Set
F1@5	0.4254	0.3716
P@5	0.2933	0.2500
R@5	0.8833	0.8500
Retrieval Score	0.5340	0.4905
Correctness (pass rate)	0.93	0.90
Faithfulness (pass rate)	1.00	1.00
Completeness	4.02/5	4.05/5
Partial Gen Score	0.9126	0.9033

Table 4: Final Pipeline Evaluation Results

Generation quality is strong for both sets. A perfect faithfulness score shows that the pipeline stays grounded in the retrieved context without hallucinating.

Golden Q&A retrieval is lower. Retrieval score on the golden set (0.49) is lower than on the validation set (0.53), likely because our self-constructed questions are more specific and require more specific/precise evidence spans that are much harder to retrieve with similarity search.

5 Surprises

We originally anticipated the cross-encoder to improve F1@5. However, we found that all three reranker configurations (MiniLM-L4, BGE-reranker, none) produce identical F1@5 within each chunk group, since rerankers only reorder the retrieved set without changing which documents are fetched.

We also anticipated that inducing diversity would reduce redundancy and yield better results. However, MMR consistently underperformed plain similarity across all chunk sizes (chunk=256: 0.3672 vs 0.3952; chunk=512: 0.3246 vs 0.4189; chunk=1000: 0.3393 vs 0.4254).

6 Future Work

If more time were available, we would explore multi-resolution chunking by indexing at multiple chunk sizes simultaneously, as classmates reported around +0.3 precision improvement. We would also test BM25 hybrid retrieval (BM25 + Pinecone) to see if it could improve retrieval quality. Finally, we would test larger chunk sizes (> 1000) to see whether providing more context per chunk improves answer completeness and hence generation quality.

7 AI Tools Statement

We used **Claude** (claude-sonnet-4.6) extensively throughout the project for debugging the RAG pipeline, identifying the a line tracking bug caused by the `str.find()` fallback in chunk metadata, diagnosing the degenerate TritonAI embedding space from diagnostic output, suggesting the switch to BGE-small embeddings, and couple of bugs in the hybrid BM25+vector architecture. All generated code was reviewed, understood, and iteratively modified.

We used `claude-sonnet-4-6` for grammar corrections, paragraph organization, and writing suggestions throughout this report. It was not used to write entire sections of the report.

We used the **RapidFire AI tutorial notebook** (`discussion1-rag-scifact.ipynb`) as a reference for correct DSMLP compatibility imports (`patch_rapidfireai_for_datahub`) and the `reranker_cfg` API in `RFLangChainRagSpec`. The experimentation code was heavily edited using the coding agent which was then subsequently reviewed and modified to align with our actual pipeline configurations and findings.

No AI tool was used to generate git commits to tamper commit history.